Machine Test Assignment – Stock Market Alert & Portfolio Tracker

Objective

Build a full-stack stock market application consisting of:

Frontend:

- Next.js

Backend:

- Node.js using any one framework:
  - Express.js
  - Fastify
  - NestJS

Database:

- MongoDB

The assignment is designed to evaluate:

- Frontend development skills
- Backend architecture
- API design
- Authentication
- Database modeling
- Asynchronous/background processing
- Clean code practices
- Project structuring

Tech Stack Requirements

Frontend

Mandatory:

- Next.js

Preferred:

- TypeScript
- Tailwind CSS

Backend

Mandatory:

- Separate backend service using Node.js

Candidate may choose:

- Express.js
- Fastify
- NestJS

Database

- MongoDB

Functional Requirements

1. Authentication

Implement:

- Signup
- Login

- Protected Routes

Fields:

- Name
- Email
- Password

Authentication can be implemented using:

- JWT
- Session-based authentication

---

2. Stock Search Module

Users should be able to:

- Search stocks by symbol
- View stock details:
  - Symbol
  - Current Price
  - Day High/Low
  - Volume
  - Percentage Change

You may use any free stock market API such as:

- Yahoo Finance
- Alpha Vantage
- Finnhub
- Any equivalent API

---

3. Watchlist Feature

Users should be able to:

- Add stocks to watchlist
- Remove stocks from watchlist
- View saved watchlist stocks

---

4. Price Alert System (Main Feature)

Users should be able to create stock alerts.

Example:

- Notify when RELIANCE price goes above ₹3000

Alert fields:

- Stock Symbol
- Condition:
  - Greater Than
  - Less Than
- Target Price

Expected Logic:

- Backend should periodically check stock prices.
- When condition matches:
  - Mark alert as triggered
  - Save triggered timestamp

You may use:

- Cron Jobs
- Background Workers
- Polling Mechanism

---

5. Portfolio P&L Module

Users should be able to:

- Add holdings:
  - Symbol
  - Quantity
  - Buy Price

Application should calculate:

- Current Value
- Profit/Loss
- Overall Portfolio Summary

Technical Expectations

Frontend Expectations

- Responsive UI
- Proper loading states
- Error handling
- Reusable components
- Proper state management
- Clean folder structure

Backend Expectations

- Proper project architecture
- REST API design
- Middleware usage

- Validation
- Error handling
- Secure password hashing
- Environment variable usage
- Separation of concerns

Database Collections

Suggested collections:

- Users
- Watchlists
- Alerts
- Holdings

Bonus Features

Optional but preferred:

- Dockerization
- Docker Compose
- Redis Caching
- WebSockets / Real-time Updates
- Debounced Search
- Unit/Integration Tests
- CI/CD
- Deployment

Clean Code & Maintainability (Important)

Code quality will be evaluated based on:

- Folder structure
- Naming conventions
- Reusable components/services
- API abstraction
- Separation of concerns
- Avoiding duplicate logic
- Consistent coding standards
- Maintainable functions/modules
- Proper Git commit history

Submission Requirements

Please submit:

1. GitHub Repository

Repository should clearly contain:

- frontend/
- backend/

1. README File including:

- Setup steps
- Environment variables
- Architecture explanation
- API details
- Assumptions/tradeoffs

1. .env.example
2. Postman Collection

If Dockerized:

- Dockerfile(s)
- docker-compose.yml

Timeline

Expected completion time:

48–72 Hours

Evaluation Criteria

Area	Weight
Clean Code & Maintainability	15%
Backend Architecture	20%
Next.js Frontend	20%
Problem Solving Ability	15%
Async/Background Jobs	10%
Database Design	10%
Documentation	5%
Dockerization	Bonus
Extra Optimizations/Features	Bonus

Important Notes

- Focus on writing clean, maintainable, and scalable code.
- Core features are mandatory.
- Bonus features are optional.
- Proper documentation and project setup are expected.